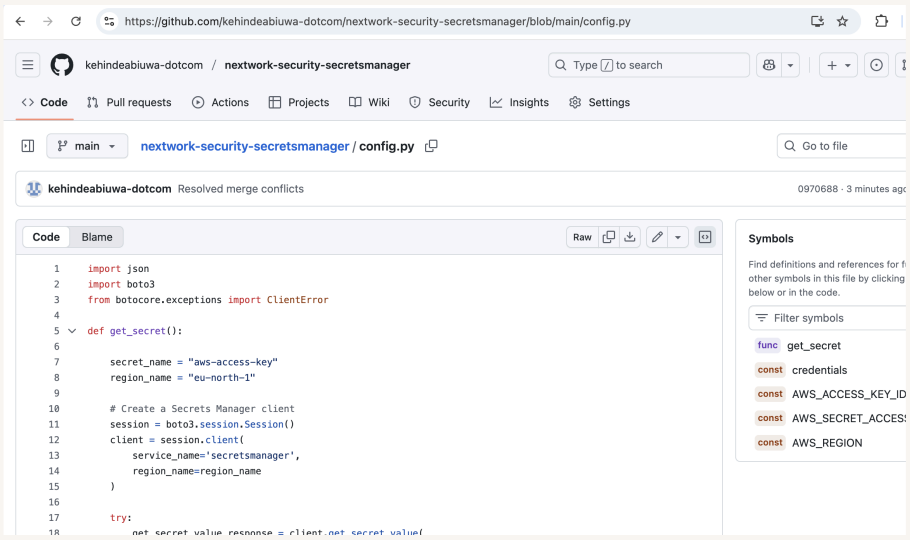
nextwork.org

Secure Secrets with Secrets Manager

Kehinde Abiuwa



Kehinde Abiuwa
NextWork Student

Introducing Today's Project!

In this project, I will demonstrate finding hard-coded credentials in a web app, seeing GitHub Secret Scanning block a risky push, then refactoring the app to read secrets from AWS Secrets Manager (via IAM/SDK) and confirming the repo can be public without exposing anything. I'm doing this project to learn how to manage secrets securely end-to-end—spot leaks early, store/rotate secrets in KMS-backed Secrets Manager, and use least-privilege access so my code and pipelines stay safe.

Tools and concepts

Services I used were AWS Secrets Manager, AWS IAM, and GitHub. Key concepts I learnt include securely storing credentials (instead of hard-coding), GitHub secret scanning & push protection, pulling secrets in code with boto3, secret rotation basics, and cleaning commit history (e.g., rebasing) to remove exposed secrets.

Project reflection

This project took me approximately 60 minutes. The most challenging part was rewriting history (interactive rebase) and resolving the merge conflict in config.py after removing the hard-coded keys. It was most rewarding to see GitHub's secret scanning block the insecure push, switch the app to pull credentials from Secrets Manager successfully, and then verify in the repo that no secrets were exposed.

K

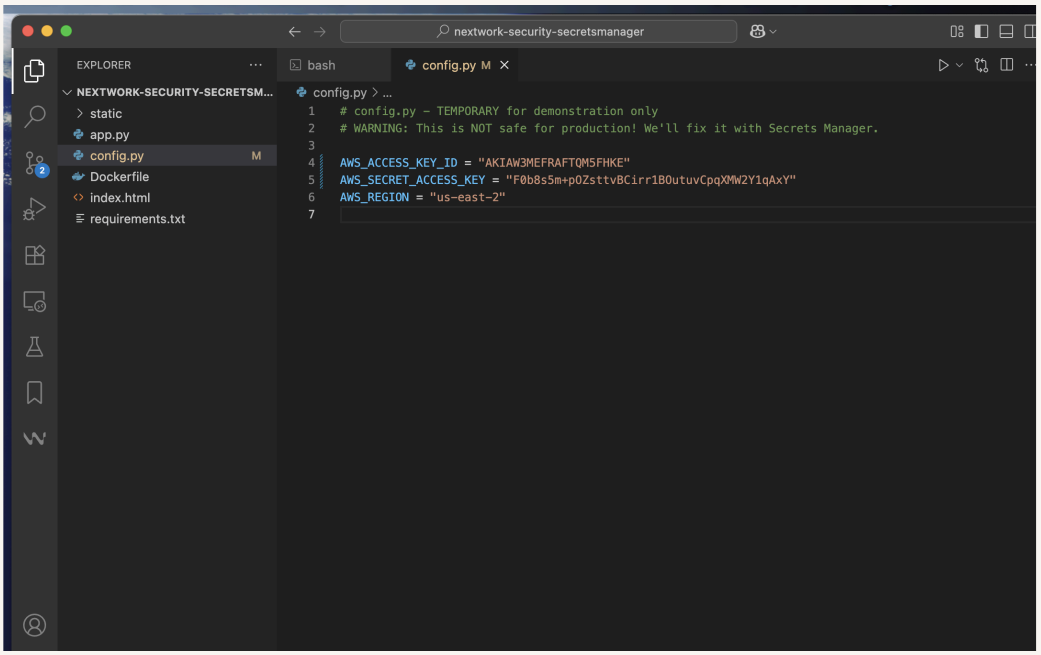
Kehinde Abiuwa
NextWork Student

I did this project to showcase my skills in my project portfolio

Hardcoding credentials

In this project, a sample web app is exposing AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY in config.py. It is unsafe to hardcode credentials because: Anyone who sees the code (or a fork/screenshot/zip) can use your keys to access your AWS account, steal data, or rack up costs. Secrets live forever in Git history, caches, and CI logs—even after you delete the line. Keys spread to every developer laptop and build system, increasing the blast radius if any one machine is compromised. Rotation is hard and error-prone; one leaked key can be reused until you notice and revoke it.

I've set up the initial configuration with hard-coded placeholders for AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY in config.py to mimic an insecure app. These credentials are just examples because I don't want to expose real keys—they won't work in AWS and are only there to demonstrate why hardcoding is risky and to trigger GitHub Secret Scanning.


nextwork-security-secretsmanager
EXPLORER
NEXTWORK-SECURITY-SECRETSM...
static
app.py
config.py
M
Dockerfile
index.html
requirements.txt
bash
config.py M
config.py > ...
# config.py — TEMPORARY for demonstration only
# WARNING: This is NOT safe for production! We'll fix it with Secrets Manager.
AWS_ACCESS_KEY_ID = "AKIAW3MEFRAFTQM5FHKE"
AWS_SECRET_ACCESS_KEY = "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"
AWS_REGION = "us-east-2"

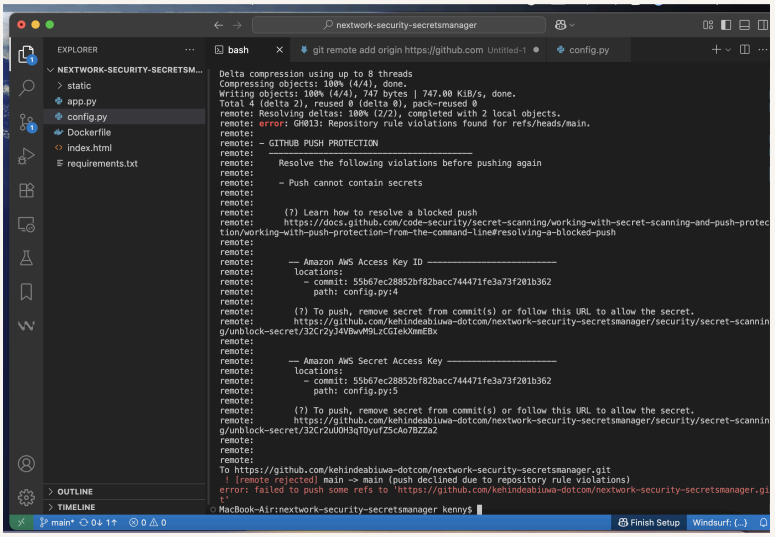
Pushing Insecure Code to GitHub

Once I updated the web app code with credentials, I forked the repository because I needed my own copy on GitHub to push to, so Secret Scanning could run and so I wouldn't affect the original repo. A fork is different from a clone in that a fork is an online copy in my GitHub account (I can push to it and make it public), while a clone is just a local copy on my computer that no one else can see unless I push it somewhere—and I wouldn't have permission to push to the original repo.

To connect my local repository to the forked repository, I ran: git init to create a new Git repo in the folder. git remote add origin <URL> to link it to my fork on GitHub, and git remote -v to verify. git remote set-url origin <URL> to fix/update the remote URL. Then I used git add . and git commit -m "..." to stage my changes (including config.py) and save them in a commit. Finally, git push -u origin main uploads the main branch to my fork and sets the upstream

GitHub blocked my push because Secret Scanning / Push Protection detected AWS credentials in my commit (config.py). It prevents pushes that contain secrets—even to public forks—to stop accidental leaks. This is a good security feature because once a secret reaches GitHub, it's instantly exposed in repo history, forks, and caches, and bots can harvest it. Blocking the push protects my account and forces me to remove/rotate the secret before sharing the code.

Kehinde Abiuwa
NextWork Student



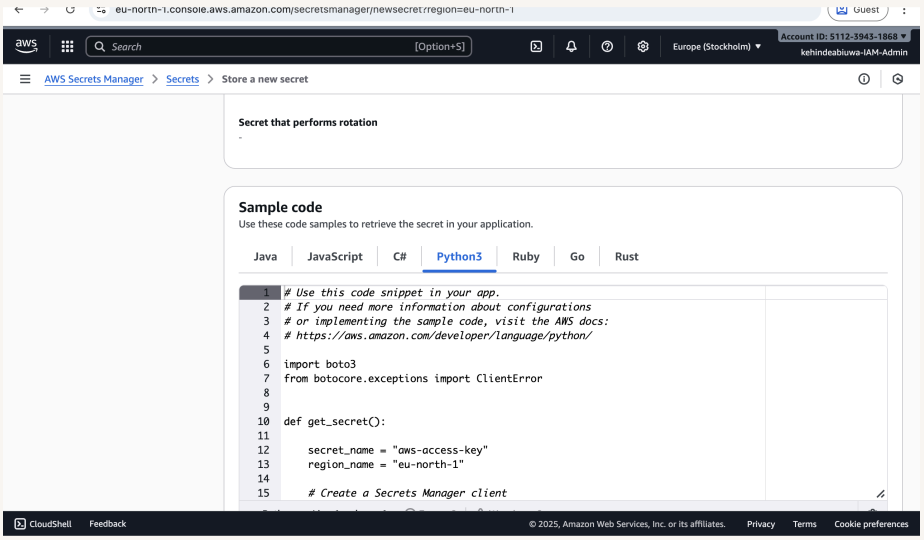
Secrets Manager

Secrets Manager is AWS's managed vault for application secrets—it stores credentials encrypted with KMS, lets you control access with IAM, audits use with CloudTrail, and can automatically rotate secrets. I'm using it to store the app's AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY as a single secret (key/value pairs), so the code can fetch them securely at runtime instead of hard-coding them in config.py. Other common use cases include database passwords (with auto-rotation for RDS/Aurora), API keys (e.g., Stripe/Twilio), OAuth tokens, webhook signing secrets, and other sensitive config like SFTP or third-party service credentials.

Another feature in Secrets Manager is Secret rotation which is the process of automatically changing your secrets on a regular schedule. This is a security best practice because it reduces the risk of compromised credentials. If a secret is compromised, it will only be valid for a limited time before it's automatically rotated. it's useful in situations where: Long-lived/static creds are needed (e.g., a database username/password or 3rd-party API key) and can't be replaced with short-lived IAM roles. Many apps/instances share the same secret, so rotating it manually would be risky or error-prone. Compliance (PCI, SOC 2, ISO) requires periodic credential changes and auditability. Vendors/contractors get temporary access you want to expire automatically. You're doing incident response and need a fast, coordinated secret change across environments.

Secrets Manager provides sample code in various languages, like Python (boto3), Node.js, Java, Go, and .NET/C#. This is helpful because you can copy a small snippet, swap in your secret name/ARN, and the AWS SDK handles auth, KMS decryption, retries, and parsing for you (there's even a caching client to avoid repeated calls). It makes replacing hard-coded credentials with runtime secret retrieval fast and low-risk.

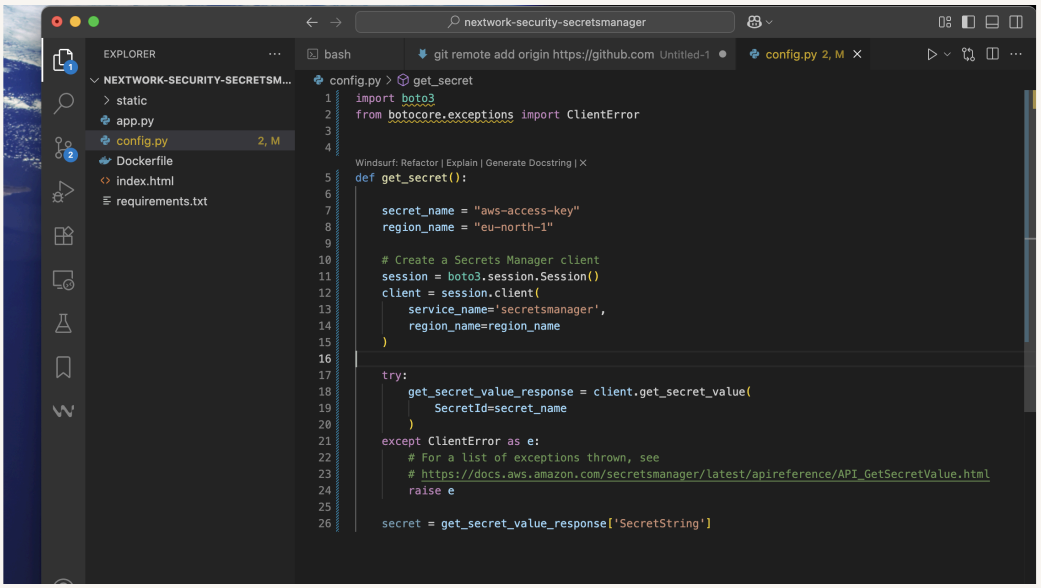
Kehinde Abiuwa
NextWork Student



Updating the web app code

I updated the config.py file to retrieve the AWS credentials from Secrets Manager, not from hard-coded strings. The get_secret() function will create a boto3 Secrets Manager client, call get_secret_value(SecretId='aws-access-key', region='eu-north-1'), read the returned SecretString JSON, and return the values (AccessKeyId/SecretAccessKey) for the app to use. This way the app pulls the keys at runtime and avoids exposing secrets in code or GitHub.

I also added code to config.py to extract the values from the secret—parsing the JSON returned by get_secret() and assigning AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, and AWS_REGION to the variables the app expects. This is important because the app now loads credentials at runtime (no hard-coding), so secrets stay in Secrets Manager, can be rotated, and aren't exposed in Git history or config files.


nextwork-security-secretsmanager
EXPLORER
NEXTWORK-SECURITY-SECRETSM...
static
app.py
config.py 2, M
Dockerfile
index.html
requirements.txt
bash
git remote add origin https://github.com Untitled-1
config.py 2, M
config.py > get_secret
import boto3
from botocore.exceptions import ClientError
Windsurf: Refactor | Explain | Generate Docstring |
def get_secret():
secret_name = "aws-access-key"
region_name = "eu-north-1"
# Create a Secrets Manager client
session = boto3.session.Session()
client = session.client(
service_name='secretsmanager',
region_name=region_name
)
try:
get_secret_value_response = client.get_secret_value(
SecretId=secret_name
)
except ClientError as e:
For a list of exceptions thrown, see
https://docs.aws.amazon.com/secretsmanager/latest/apireference/API_GetSecretValue.html
raise e
secret = get_secret_value_response['SecretString']

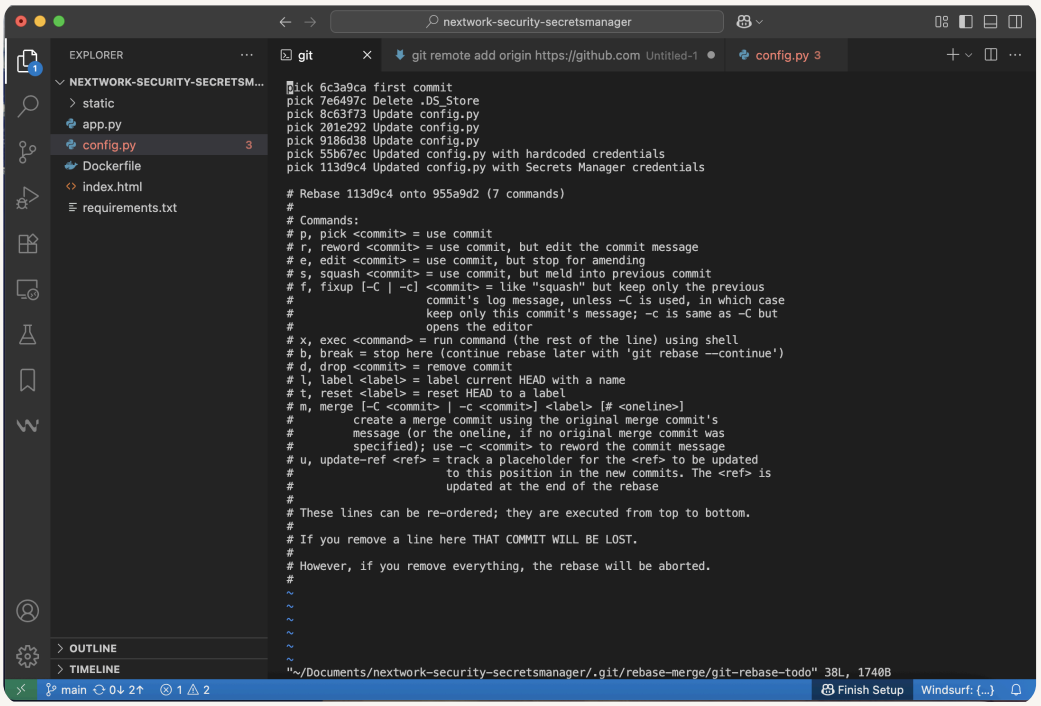
Rebasing the repository

Git rebasing is rewriting a branch's history by replaying your commits onto a new base, where you can edit, reorder, squash, or drop commits. I used it to rewrite my local history and delete/modify the earlier commit that hard-coded AWS keys. This was necessary because the keys still existed in the commit history, and GitHub Push Protection scans the entire history in the push. Rebasing let me remove the bad commit so the secret is purged and the push can proceed safely.

A merge conflict occurred during rebasing because two commits edited the same lines in config.py—one with hard-coded keys and the later one with the Secrets Manager code. I resolved the merge conflict by opening config.py, keeping the Secrets Manager version, and deleting the conflict markers (<<<<<<< HEAD, =======, >>>>>>> ...) and the old hard-coded section. Then I saved the file, ran git add config.py.

Once the merge conflict was resolved, I verified the secrets were gone by opening my fork on GitHub in the browser and viewing config.py on main. I could see only the Secrets Manager code (get_secret() with boto3) and no plaintext AWS keys, and my push was no longer blocked by GitHub's secret scanning. I also glanced at the commit history to confirm the old commit with hardcoded credentials was removed by the rebase.

Kehinde Abiuwa
NextWork Student



The place to learn & showcase your skills

Check out nextwork.org for more projects